Classification of Legal Text: US Supreme Court Opinions

Machine Learning Mini-Project

Course Code : UE24CS352A

Team Number : **C-26** | Section : **AIML-C**
Team Members : **Kshitij Satish Shetty (PES1UG24AM143)**
: **Kushal G (PES1UG24AM145)**
GitHub repository**:** *https://github.com/KshitijShetty27/Legal-Text-Classification_ML_Mini_Project*

1. Problem Statement

Given the text of a US Supreme Court opinion, predict its issue area, the legal topic assigned by expert coders in the Supreme Court Database (for example Criminal Procedure, Civil Rights or Federal Taxation). This is multi-class text classification on long, formal documents. Cataloguing opinions by hand is slow, so a reliable automatic classifier helps search and legal research. The assigned reference is the CS229 (2020) report by K. Iyer, which uses the same data.

2. Dataset

The Supreme Court Database as packaged by the textacy project: 8,419 opinions decided between 1946 and 2016, each with full text and expert labels; the target is issue_area. Opinions are long (median about 5,550 words, maximum 87,246) and the classes are imbalanced (Criminal Procedure 1,924 documents, Miscellaneous 19). Cleaning removed 91 rows (texts under 500 characters, unlabeled rows, exact duplicates and the one-document class Private Action), leaving 8,328 documents in 13 classes. Only the first 30,000 characters of each opinion are used to keep CPU time and memory low. A single stratified split gives 5,829 train, 1,249 validation and 1,250 test documents (seed 42).

3. Approach

- **Features:** TF-IDF on single words (English stop-words removed, minimum document frequency 5, sub-linear term frequency, 31,568 terms kept), fitted on the training split only.
- **Models:** majority-class baseline; Logistic Regression (C=1, 10); Linear SVM (C=0.1, 0.5, 1); Complement Naive Bayes (alpha=0.1, 0.5); and the two models of the reference report re-implemented on our split: LDA (25 topics) + Logistic Regression and Doc2Vec (PV-DBOW, 100-d, first 10,000 characters) + Logistic Regression. Class weights are balanced.
- **Selection:** by macro-F1 on the validation set, so every class counts equally; the test set was used once at the end. Best model: Linear SVM, C=0.5.
- **Leakage control:** split before any fitting; TF-IDF, LDA and Doc2Vec learn from training data only; no identical text appears in two splits; a shuffled-label test gave validation accuracy 0.167, below the 0.231 majority rate, so the features alone do not reveal the label.
- **Robustness checks:** 5-fold cross-validation (TF-IDF re-fitted inside each fold), a fine-grained issue-code task, error analysis and a similar-opinion search.
-

4. Implementation Overview

Everything runs on CPU in one Google Colab notebook: data download, cleaning, split, features, model comparison, evaluation, explainability, extra experiments and model export. The best pipeline (TF-IDF + Linear SVM, 3.4 MB) is saved with joblib and trains in about 3 seconds; LDA took about 2.5 minutes and Doc2Vec about 7 minutes. A Streamlit app (app.py) loads the pipeline, lets the user paste an opinion or pick a held-out test opinion, and shows the top three issue areas with scores and the words that pushed the prediction most. The repository holds the notebook (with outputs), app.py, the saved model, result files, requirements.txt and a README with run instructions.

5. Results

Model	Val acc	Val macro-F1	Test acc	Test macro-F1	Test weighted-F1
Majority-class baseline	0.2306	0.0288	0.2304	0.0288	0.0863
LDA (25 topics) + Logistic Regression	0.5540	0.4457	0.5816	0.4472	0.6118
Doc2Vec (100-d) + Logistic Regression	0.6757	0.6059	0.6696	0.5843	0.6761
TF-IDF + Linear SVM (C=0.5) [selected]	**0.7958**	**0.7322**	**0.8176**	**0.7672**	**0.8156**

Table 1. Validation and test performance. Other validation results: Logistic Regression C=10 accuracy 0.7846 / macro-F1 0.7107; best Complement NB 0.7222 / 0.5725.

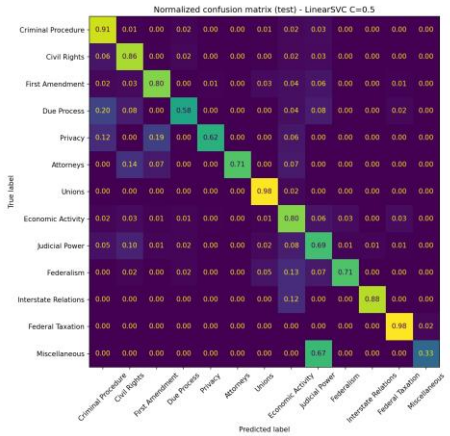


Fig. 1. Normalized confusion matrix (test set).

- **Strongest:** Criminal Procedure (F1 0.89), Federal Taxation (0.88), Unions (0.86, recall 0.98), Civil Rights and First Amendment (0.84).
- **Weakest:** Miscellaneous (0.40, only 3 test documents), Due Process (0.61), Judicial Power (0.70), Privacy (0.71).
- **Errors:** 228 of 1,250 test opinions were misclassified, mostly between legally adjacent areas: Judicial Power to Civil Rights (16), Economic Activity and Judicial Power (14 each way), Civil Rights to Criminal Procedure (12), Due Process to Criminal Procedure (10).
- **Interpretability:** top-weighted words are sensible (jury, search, habeas; labor, union, nlrb; tax, taxpayer, refund), but some are case names or numbers (bruton, garmon, chadha, 1988), so the model partly leans on case-specific vocabulary.

Cross-validation. Over 5 folds of train + validation, Linear SVM scored accuracy 0.808 ± 0.020 and macro-F1 0.746 ± 0.027, so the single-split result (0.818 / 0.767) is not a lucky split.

Fine-grained task. Predicting the specific issue code (141 codes with at least 15 opinions, 7,434 opinions, 80/20 split) gives accuracy 0.707, top-5 accuracy 0.931 and macro-F1 0.656, against 0.032 for always predicting the majority code.

Similar opinions. As in the reference report, document vectors work as a search index. The nearest opinions to Roe v. Wade (1973) are mostly Privacy-area cases; Doe v. Bolton (1973) and Planned Parenthood v. Danforth (1976) appear in both the Doc2Vec and the TF-IDF lists.

6. Comparison with the Reference Report

Iyer reports about 0.47 accuracy for LDA + Logistic Regression and 0.63 for Doc2Vec + Logistic Regression on 15 issue areas (we read Table 1 using the report's text, because its column headings appear swapped), and BERT ran out of memory. On our 13-class split LDA + Logistic Regression reaches 0.58, Doc2Vec + Logistic Regression 0.67 and TF-IDF + Linear SVM 0.82. The comparison is indicative only: the label set, cleaning, split and libraries differ. The paper's ordering (Doc2Vec above LDA) is reproduced, and a simple supervised bag-of-words model beats both by a wide margin.

7. Conclusions and Limitations

- TF-IDF + Linear SVM is the best model: 0.82 test accuracy and 0.77 macro-F1 (baseline 0.23 and 0.03). It is CPU-only, trains in seconds and is explainable through word weights.
- Unsupervised features (LDA topics, Doc2Vec vectors) lose label-specific information that supervised word weights keep.
- Limitations: a single held-out test split (cross-validation shows the spread); classes with 3 to 16 test documents give noisy per-class scores; only single words from the first 30,000 characters are used; case-specific words inflate some class weights; no transformer was trained. Next steps: word pairs, a time-based split, and Longformer or Legal-BERT on a GPU.